

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדוראלי

79115 – מבוא למדעי המחשב - תלפיות

תאריך: 20.2.2009
משך הבחינה: שלוש שעות

מועד א' - סמסטר א' - תשס"ט – 2008-2009
פרופ' ג'ף רוזנשיין

הנחיות:

1. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
2. יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
3. יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
4. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תוכה את כותבה במלוא הנקודות.
5. יש למחוק טיוטות באופן ברור.
6. יש להשאיר שוליים.
7. אסור להשתמש בעט אדום.
8. הקפידו על סגנון תכנותי נאותו כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
9. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
10. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
11. אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
12. בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.

שאלה 1 (33 נקודות)

נתון הממשק Merger עבור אובייקטים היודעים למזג שני אובייקטים נתונים לאובייקט אחד:

```
public interface Merger {  
    public Object merge(Object obj1, Object obj2);  
}
```

נתון הממשק Resistor עבור אובייקטים בעלי התנגדות חשמלית:

```
public interface Resistor {  
    public double resistance();  
}
```

א. (4 נק')

כתבו שיטה סטטית המשתמשת באובייקט Merger כדי למזג מערך של אובייקטים לאובייקט אחד ע"פ הסדר (האובייקט הראשון ממוזג עם האובייקט השני, התוצאה ממוזגת עם האובייקט השלישי, וכו'):

```
public static Object mergeArray(Object[] objects,  
                                Merger merger)
```

ב. (4 נק')

כתבו מחלקה SimpleResistor המממשת (implements) את הממשק Resistor ומייצגת נגד חשמלי יחיד בעל התנגדות כלשהי. חתימת בנאי (constructor) המחלקה צריכה להיות:

```
public SimpleResistor(double resistance)
```

ג. (5 נק')

כתבו מחלקה אבסטרקטית CompoundResistor המממשת גם היא את הממשק Resistor ומייצגת רכיב המורכב משני תתי-רכיבים בעלי התנגדות. (ההתנגדות של רכיב כזה תלויה בצורת החיבור של תתי-הרכיבים, כמפורט בסעיף הבא). חתימת בנאי המחלקה צריכה להיות:

```
public CompoundResistor(Resistor resistor1,  
                        Resistor resistor2)
```

ד. (8 נק')

כתבו שתי מחלקות SerialResistor, ParallelResistor (היורשות extends) מהמחלקה CompoundResistor ומייצגות הרכבה של שני רכיבים בעלי התנגדות בטור או במקביל, בהתאמה. חיבור של רכיבים בעלי התנגדויות R1, R2 בטור יוצר רכיב בעל התנגדות R1+R2. חיבור של רכיבים בעלי התנגדויות R1, R2 במקביל יוצר רכיב בעל התנגדות $(R1 \cdot R2) / (R1 + R2)$.

ה. (6 נק')

כתבו שתי מחלקות `SerialResistorMerger`, `ParallelResistorMerger` המממשות את הממשק `Merger` וממוגות שני רכיבים בעלי התנגדות ע"י חיבורם בטור או במקביל, בהתאמה. מותר להניח שבכל קריאה לשיטה `merge` במחלקות הנ"ל מועברים כפרמטרים אובייקטים המממשים את הממשק `Resistor`.

ו. (6 נק')

כתבו שתי שיטות סטטיות המחברות מערך של רכיבים בעלי התנגדות בטור או במקביל:

```
public static Resistor connectSeries(Resistor[]  
resistors)  
  
public static Resistor connectParallel(Resistor[]  
resistors)
```

על השיטות הנ"ל להשתמש בשיטה שכתבתם בסעיף א'. להלן דוגמה לקטע תוכנית המשתמש בשיטות הנ"ל:

```
SimpleResistor[] r1 = {new SimpleResistor(2.0),  
new SimpleResistor(6.0)};  
SimpleResistor[] r2 = {new SimpleResistor(3.0),  
new SimpleResistor(1.0),  
new SimpleResistor(2.0)};  
System.out.println(connectParallel(r1).resistance());  
System.out.println(connectSeries(r2).resistance());
```

הפלט של קטע זה הוא:

1.5

6.0

שאלה 2 (33 נקודות)

רקורסיה - נורות

למשה-תאורה-ואווירה בע"מ יש ערכת תאורה מעט מוזרה. בערכה שורה של נורות, כל נורה יכולה להיות במצב דלוק או כבוי. לכל נורה צמוד מתג המשנה את מצב הנורה ואת מצב שתי הנורות הסמוכות לה - אחת מימינה ואחת משמאלה (או הנורה היחידה הסמוכה לה אם המתג הוא של נורה קיצונית). למשל, עבור שורת הנורות הזו **0xxx0x** (0 - נורה דולקת, ו-x - נורה כבויה), לחיצה על המתג האמצעי תשנה את מצב שורת הנורות ל- **000xx**.

כתבו שיטה רקורסיבית

```
public static boolean disco(boolean[] init, boolean[] target)
```

המקבלת שני מערכים בוליאניים שווים באורכם - הראשון מייצג את מצב הנורות ההתחלתי והשני מייצג את מצב הנורות הרצוי. השיטה תחזיר **true** ותדפיס את סדרת מספרי המתגים עליהם יש ללחוץ על מנת להעביר את הנורות מהמצב ההתחלתי למצב הרצוי. השיטה תחזיר **false** ולא תדפיס דבר אם לא ניתן להגיע למצב היעד.

דוגמה:

עבור הקלט:

```
boolean[] init = {true, false, true, false, true, false};
boolean[] target = {false, true, false, true, false, true};
disco(init, target);
```

פלט אפשרי יהיה:

1

4

רמז: שימו לב שלחיצה על מתג פעמיים לא משנה דבר ושסדר הלחיצות אינו משנה.

שאלה 3 (33 נקודות)

Ternary Search

אין לפתור שאלה זו ברקורסיה.

נתון מערך של מספרים שלמים ממוינים בסדר לא יורד (כלומר מערך ממוין בסדר עולה פרט לערכים חוזרים). בשאלה זו נממש אלגוריתם חיפוש ternarySearch למציאת ערך נתון במערך הממוין. בכל שלב באלגוריתם, מחלקים את קטע המערך הנותר לשלושה חלקים שווים (עד כמה שניתן), וע"י בדיקה של לכל היותר שני תאים במערך מחליטים באיזה שליש נמצא הערך המבוקש. לאחר מכן מחלקים את שליש המערך הנבחר לשלושה חלקים שווים, וכך הלאה, עד שהערך המבוקש נמצא (או עד שברור כי הערך לא קיים במערך).

לדוגמה, נניח כי עלינו לחפש את הערך 23 במערך הבא:

-12	17	18	23	27	31	40	52	52
-----	----	----	----	----	----	----	----	----

נחלק את המערך לשלושה חלקים (כל חלק מכיל שלושה תאים). ראשית, נשווה את הערך המבוקש לתא האחרון בשליש הראשון. מכיוון ש- $18 < 23$, אנו יודעים כי הערך המבוקש לא יימצא בשליש הראשון. כעת נשווה את הערך המבוקש לתא האחרון בשליש השני. מכיוון ש- $23 < 31$, אנו יודעים כי עלינו להמשיך את החיפוש בשליש השני. לשם כך נחלק גם את השליש השני של המערך לשלושה חלקים. נשווה את הערך המבוקש לתא היחיד בשליש הראשון – ומצאנו את מבוקשנו. ערך ההחזרה של האלגוריתם הוא האינדקס במערך שבו נמצא הערך המבוקש, בדוגמא שלנו יוחזר האינדקס 3.

א. (15 נק')

כתבו שיטה `public static int ternarySearch(int[] data, int key)` המקבלת מערך `data` ממוין בסדר לא יורד וערך מבוקש `key`, ומחזירה את האינדקס של הערך המבוקש במערך באמצעות האלגוריתם שתואר לעיל. אם הערך המבוקש אינו נמצא במערך, יש להחזיר -1. אם הערך המבוקש מופיע יותר מפעם אחת במערך, יש להחזיר את האינדקס של אחד ממופיעיו. על השיטה לעבוד עבור מערך בכל גודל (אין להניח שמספר התאים במערך מתחלק בשלוש).

ב. (5 נק')

כתבו שיטה `public static boolean findSum(int[] data, int z)` המקבלת מערך `data` שאינו ממוין ומספר `z`, ומחזירה `true` אם ורק אם קיימים במערך שני מספרים `x, y` כך ש- $x+y=z$. על השיטה לרוץ בסיבוכיות זמן $O(n^2)$ במקרה הגרוע (כאשר `n` גודל המערך). הראו שעמדתם בדרישה זו.

ג. (13 נק')

כתבו שיטה

`public static boolean findSumSorted(int[] data, int z)`

המקבלת מערך `data` ממוין בסדר לא יורד ומספר `z`, ומחזירה `true` אם ורק אם קיימים במערך שני מספרים `x, y` כך ש- $x+y=z$. על השיטה לרוץ בסיבוכיות זמן $O(n^2)$ במקרה הגרוע (כאשר `n` גודל המערך). הראו שעמדתם בדרישה זו. ניתן להשתמש בשיטה המתוארת בסעיף א', גם אם לא פתרתם את סעיף א'.

שאלה 4 (33 נקודות)

מימוש strand sort על רשימות מקושרות:

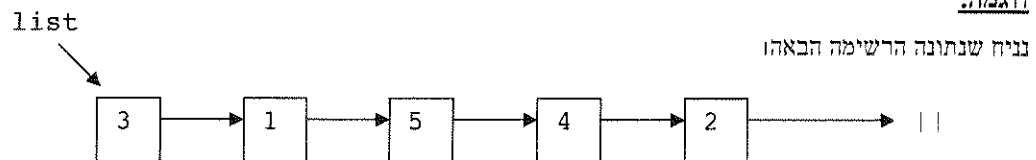
האלגוריתם **strand sort** הינו אלגוריתם מיון איטרטיבי. בשאלה זו, נדרש לממש אלגוריתם זה על מנת למיין רשימה מקושרת של מספרים.

בכל איטרציה של האלגוריתם, מחלצים מהרשימה המקורית תת-רשימה ממוינת (לאו דווקא רציפה) וממזגים אותה לתוך רשימת התוצאה.

להלן שלבי האלגוריתם:

1. אתחל את רשימת התוצאה לרשימה ריקה.
2. עבור על הרשימה המקורית וחלץ ממנה את תת-הרשימה הממוינת המתחילה באיבר הראשון.
3. מזג את תת-הרשימה הממוינת לתוך רשימת התוצאה (לקבלת רשימת תוצאה חדשה וממוינת).
4. חזור על שלבים 2-3 עד שהרשימה המקורית ריקה.

דוגמה:



להלן טבלה המכילה את ערכי הרשימות, בכל שלב באלגוריתם המיון:

רשימה מקורית	תת-הרשימה הממוינת	רשימת התוצאה
3 1 5 4 2		
1 4 2	3 5	
1 4 2	3 5	
2	1 4	3 5
2		1 3 4 5
	2	1 3 4 5
		1 2 3 4 5

נתונה המחלקה הבאה לייצוג חוליה ברשימה מקושרת:

```

public class Node {
    public int getData()
    public Node getNext()
    public void setNext(Node next)
    public static Node merge(Node list1, Node list2)
}
  
```

כאשר השיטה הסטטית **merge** ממזגת שתי רשימות ממוינות לרשימה ממוינת אחת (אין צורך לממש מחלקה זו).

א. (27 נק')

עליך לממש שיטה לא רקורסיבית

```
public static Node strandSort(Node list)
```

המקבלת רשימה מקושרת (בלתי ממוינת), ומחזירה רשימה המכילה את איברי הרשימה המקורית ממוינים בסדר לא יורד (כלומר בסדר עולה פרט לערכים חוזרים), תוך שימוש באלגוריתם שתואר לעיל. אינך נדרש לשמור על שלמות הרשימה `.list`.

ב. (3 נק')

מהי סיבוכיות זמן הריצה במקרה הגרוע ביותר של השיטה `strandSort`? מהו המקרה הגרוע ביותר? הסבר!

ג. (3 נק')

מהי סיבוכיות זמן הריצה במקרה הטוב ביותר של השיטה `strandSort`? מהו המקרה הטוב ביותר? הסבר!

ב ה צ ל ח ה !